MEDICAL PRESCRIPTION AI

PROJECT REPORT [Phase-I]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



Submitted by

Pradeep Chahar (24U020117, 5th Semester)
Alok Kumar (24U023023, 5th Semester)

Under the supervision of

Supriyo Mandal
Assistant Professor
(Department of Computer Science and Engineering)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled “Medical Prescription AI” has been satisfactorily completed by Pradeep Chahar (24U020117) and Alok Kumar (24U023023). It is a bonafide piece of work, carried out under our guidance in the Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal for the partial fulfillment of the Bachelor of Technology during the academic year 2026-27.

Date:

Dr. Supriyo Mandal
Assistant Professor,
Department of CSE
IIIT Bhopal (M.P)

Dr. Rahul Chaurasia
Head / Department Coordinator,
Department of CSE
IIIT Bhopal (M.P)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

We hereby declare that the project report entitled “Medical Prescription AI” is presented in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering. It is an authentic documentation of our original work carried out under the able guidance of Supriyo Mandal, Assistant Professor. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Date:

Pradeep Chahar (24U020117) Sign of student1

Alok Kumar (24U023023) Sign of student2

ACKNOWLEDGEMENT

We, Pradeep Chahar and Alok Kumar, would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our project (Phase-I) titled “Medical Prescription AI”.

First and foremost, we would like to extend our heartfelt thanks to our esteemed **Director**, Dr. Ashutosh Kumar Singh, for providing us with the necessary resources and facilities for conducting our project. The encouragement and vision of the Director played a key role in helping us accomplish our goals.

We are deeply grateful to our **Project Guide**, Supriyo Mandal, Assistant Professor, for the constant guidance, valuable suggestions and expert advice throughout the project. The knowledge and insights shared with us helped us improve the quality of our work, and the support kept us motivated during difficult times.

Our sincere thanks to the Department coordinator Dr. Rahul Chaurasiya, for giving us the support and encouragement provided during the completion of the project.

We would also like to express our thanks to all the **faculty members** of the Department of CSE for their valuable guidance and support in solving doubts and giving us with the right direction throughout the our project.

We are also thanks to our friends for giving a continuous support and help us in every problem that I have face during my project.

Last but certainly not the least, we would like to express our deepest gratitude to our **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Pradeep Chahar 24U020117,
Alok Kumar 24U023023

AREA OF WORK

Our project focuses around NLP (Natural Language Processing) such as openCV, paddleOCR and Web Devlopment . In this OCR read prescription images and extract the text, a LLM to arrange the these text into desired formate, and approximate string matching to check medicine names. Whilst this task can be carried out by just one tool such as an OCR or just a language model, using an OCR by itself will give us just a list of lines of text. Whilst using a language model may try to predict some text that isn't present within the image. We therefore include this and ensure we have some verification of the medicine names. Medicine is one area where small errors in reading can cause large issues. Many prescriptions are still written on paper so tools that can accurately digitise them whilst also being honest about their uncertainties can help within this field with patients, pharmacists and students. In our project we look at the design of a pipeline that connects a React website, a Node.js and Express server, a Python AI server (OpenCV, PaddleOCR, Gemini and RapidFuzz) and a MongoDB database, and we study how well each part works on sample prescriptions.

TABLE OF CONTENT

S.no	Title	Page No.
1	Certificate	i
2	Declaration	ii
3	Acknowledgement	iii
4	Area of Work	iv
5	List of Figures	vi
6	List of Tables	vii
7	Abstract	1
8	1. Introduction	2
9	2. Literature Review	4
10	3. Problem Definition and Objectives	7
11	4. Proposed Methodology and Work Description	8
12	5. Proposed Algorithms	13
13	6. Proposed Flowchart / Block Diagram / DFD	15
14	7. Implementation (Tools and Technology Used)	17
15	8. Result Discussion and Analysis	20
16	9. Conclusion and Future Scope	30
17	References	32
18	Plagiarism Report (Turnitin)	34

LIST OF FIGURES

Fig	Description	Page no.
1	Architecture of Medical Prescription AI	8
2	Medicine name checking with RapidFuzz	10
3	Data model of a saved prescription	11
4	Flowchart of prescription processing	15
5	Data flow diagram, Level 0	16
6	Data flow diagram, Level 1	16
7	A sample prescription before and after preprocessing	21
8	Landing page of the application	24
9	Upload page	24
10	Result page	27
11	Prescription history page	27

LIST OF TABLES

Table No	Description	Page no.
1	Comparison of related work and proposed system	6
2	State of the planned work at the end of Phase-I	7
3	Datasets and the name lists made from them	10
4	Prescription fields that will be stored	12
5	Tools and technologies used	17
6	Preprocessing task and parameters	18
7	System flow	19
8	Testing environment	20
9	OCR lines in a sample handwritten prescription	22
10	Results of the testing Medicine Name Verification	22
11	Comparative Features of the Approaches	28
12	Accuracy and Time taken	28

ABSTRACT

Reading a medical prescription is not always easy, especially when doctors write medicine names by hand. Sometimes, even a small mistake in understanding a medicine name or its dosage can create confusion for patients. To address this problem, we developed a web-based application called Medical Prescription AI. It allows users to upload a prescription and converts the information written on it into a more organised and readable format. The application works through three main components. First, the user uploads a prescription image through a website developed using React. The uploaded file is then handled by a Node.js and Express backend, which forwards it to the Python-based AI server for processing. Before extracting the text, the image is improved using OpenCV techniques such as grayscale conversion, contrast enhancement and sharpening. PaddleOCR is then used to recognise the text present in the prescription. Since the recognised text may contain spelling errors, particularly in medicine names, a Gemini language model is used to organise the extracted information into meaningful fields, including patient details, doctor information, medicine names, dosage, frequency, duration and instructions. Another important feature of the application is medicine name verification. The extracted medicine names are checked against two medicine datasets containing 16,449 product names and 1,305 generic medicine names. For this purpose, we have used the RapidFuzz library to identify similar spellings and display possible corrections. This helps users notice potential OCR mistakes instead of relying entirely on the extracted text. The processed prescription details are also stored in MongoDB, allowing users to access their previously uploaded prescriptions through a history page. 1 Currently, the project has reached Phase-I, with a functional prototype available. During initial testing with 18 typed medicine names, the medicine verification module produced correct decisions in 15 cases. The remaining cases helped us identify some limitations in the current approach, which are discussed in this report. The application is intended to assist patients in understanding their prescriptions. It only provides possible medicine name corrections and requires users to verify the information. It is not designed to replace the advice of a qualified doctor or pharmacist

Keywords: *Prescription OCR, PaddleOCR, OpenCV, Gemini, RapidFuzz, MongoDB, React, Node.js*

1. INTRODUCTION

Although the prescription is a small document, it provides a important set of information to the pharmacist. The pharmacist uses the prescription to know which medicines will be given and how many and at what time. Many hospitals and clinics in India still have prescriptions written manually onto plain paper or printed pads. This can vary greatly between doctors and include medical abbreviations and brand names.

Patient may get in trobble if the medicine read wrong . Medication errors are known to be a serious and preventable issue for patient safety [1,2]. One known reason for medication errors is unclear communication. Even if the medicine is read correct it also need to store, search and categorise the data of prescription. Therefore, the patient must search for the paper.

However, OCR have change and improved over the time and now It is better to extract the medicine from prescription, and large language models can unstructured text and produce a structured document. We wanted to investigate how far we could go with combining these two technologies to solve a real world problem. A prescription is a great example as it includes both printed and handwritten text, numerous spelling errors post OCR and there is a clear requirement for accurate information.

We noted that OCR and AI models on their own would not be sufficient. OCR produces a list of text lines. Whilst the language model can structure the information, it could be guess work. Therefore we decided to create a pipeline with clear steps. The final step would verify the medications names against an actual list.

The Medical Prescription AI works in simple steps. The user uploads an image of the prescription and waits a few seconds whilst the website extracts the text. The website will display the extracted text, patient and doctor names along with a list of medications including dosage, frequency and duration. If the medication appears to have a spelling error, the original name and suggested spelling will be displayed along with a similarity score. At the top of the results page there is a warning to double check everything against the original paper. All prescriptions processed will be stored and can be accessed at a later date from the history page.

During Phase-I we created and linked all key elements of the pipeline to create a functional prototype. Features completed include: uploading an image of a prescription, processing the image using OpenCV, extracting text using PaddleOCR, structuring using Gemini, matching medications names using RapidFuzz and medication datasets, storing prescription history using MongoDB and creating a web application using React, Node.js and Express. Features such as logging in as a user, editing results and testing against a large set of prescriptions will be completed during the next phase. These are included in the future scope.

Chapter 2 reviews the earlier work that is related to our project. Chapter 3 states the problem and the objectives. Chapter 4 explains the methodology and Chapter 5 gives the algorithms. Chapter 6 shows the block diagram, flowchart and data flow diagrams. Chapter 7 describes the tools and how the system is implemented. Chapter 8 presents the results and discussion, and Chapter 9 gives the conclusion and future scope. The references are given at the end.

The objectives of Phase-I are:

•To build a website where a user can upload a prescription image and see the result.

• To improve the quality of the uploaded image with OpenCV before reading it.

• To extract the text from the image using PaddleOCR.

• To convert the raw text into structured fields (patient, doctor, medicines, dosage, frequency, duration and instructions) using a Gemini model, without inventing missing information.

• To check medicine names against medicine datasets using RapidFuzz and show a suggested spelling with a similarity score.

• To store every processed prescription in MongoDB and show it again in a history page

2. LITERATURE REVIEW

This chapter looks at the work that is close to our project. We start with why prescription reading matters, then go through OCR, the earlier attempts on prescriptions, image preprocessing, language models, and finally approximate string matching, which we use to check medicine names.

The Institute of Medicine report "To Err Is Human" brought wide attention to the fact that mistakes in healthcare are common and that many of them can be prevented [1]. Medication errors are a large part of this. The World Health Organization later made "Medication Without Harm" a global patient-safety challenge and set the target of reducing severe, avoidable medication-related harm [2]. Poorly written prescriptions are one of the ways in which information gets lost between the doctor, the pharmacist and the patient. This is the main reason where a tool is needed to extract the information and flag unclear and doubtful medicine name is needed.

OCR tools used to be very common, it uses a multi-stage approach to detect lines, segment words and characters, then classify [3]. These perform well on clean printed documents but poorly on images, varying lighting and handwriting.

With deep learning, the CRNN model was introduced. This uses a convolutional network (image analysis) and a recurrent network (sequence reading). It can be trained fully end-to-end with no need to segment the characters [4]. Connectionist Temporal Classification (CTC) allows these models to be trained when you do not know the exact location of each character [5]. This was extended to handwritten text, and recurrent networks provided excellent results on unconstrained handwritten text [6].

Current OCR toolkits use two models for text detection and text recognition. Text detection is performed by DB (Differentiable Binarization) which is quick and performs well with varying shapes and sizes of text [7]. PP-OCR combines this with a lightweight text recognition model and text orientation classifier to produce a working system [8]. The latest version, PaddleOCR 3.0, uses PP-OCRv5 for text recognition. This shows that models with less than 100 million parameters can provide similar accuracy to large models [9]. We chose to use PaddleOCR as it is free, lightweight and can be run on a normal laptop with no GPU.

Transformer based recognisers have also appeared. TrOCR uses an image Transformer as the encoder and a text Transformer as the decoder, and it gave very good results on printed, handwritten and scene text [10]. Such models are strong, but they usually need fine-tuning on handwriting data, which we do not have in Phase-I.

Several researchers have worked on reading medicine names from prescriptions. Paul et al. noticed that normal OCR makes many errors on handwritten prescriptions, so they first found the lines that contain medicine names and then added a medicine-specific language model trained on synthetic data; their method extracted medicine names more than 2.5 times better than ready-made OCR systems [11].

Ali et al. used Mask R-CNN to cut out the medicine part of a handwritten prescription and a TrOCR model with multi-head attention to read it, and then matched the text against a database

of medicine names; they report a character error rate of 1.4% [12]. It is worth noting that this work also ends with a matching step against known medicine names, just like ours.

Mankash et al. used Indian prescriptions to fine tune some multimodal models (Qwen-VL, LLaVA and Idefics2) using 743,118 synthetic medical record images generated by 1,133 doctors. They found an accuracy of 82% for extracting medicines and dosages [13]. This shows that interpreting handwriting remains challenging despite using huge training data.

Whilst these papers provide good accuracy, they rely on specific training data and large models. In our case it wasn't realistic to try to retrain a model for our student project. We were therefore looking for a solution that would use off the shelf OCR, a generic language model and a list of medicines. The real challenge was ensuring these could be combined in a safe way to create a working web application.

Images taken using a phone will often have issues like shadows, low contrast and grey paper. Preprocessing tries to overcome this. By applying histogram equalisation the brightness values are spread out to make it easier to see the faint text. The adaptive method ensures that only small areas are considered, meaning any shadows in one corner of the image won't affect the rest [14]. The CLAHE method also limits how much noise can be enhanced [15]. A very common preprocessing step is thresholding, for example Otsu's method. This makes everything in the image completely black/white [16]. All of these techniques are easily available through OpenCV [17]. In our project we chose to convert to greyscale, apply CLAHE and sharpen. We didn't force the image into black and white, instead keeping the grey levels

The Transformer architecture [18] made it possible to train large language models that follow instructions given in plain English. Gemini is a family of such models from Google that can also work with images and long inputs [19]. Studies on medical question answering show that large language models contain a good amount of clinical knowledge, though they are not perfect [20].

The main danger of using a language model is hallucination, which means writing something that sounds right but is not in the source [21]. For prescriptions this is a serious risk, because an invented medicine name is worse than a missing one. One common way to reduce the problem is to give the model trusted reference information instead of letting it depend only on its memory [22]. We follow the same spirit in two ways. First, our prompt tells the model to only organise what is in the OCR text, to use null when something is missing and "uncertain" when something is unclear. Second, we check every medicine name against a real medicine list afterwards.

To find the correct spelling of a damaged word, we need to measure how close two strings are. The Levenshtein distance counts the smallest number of insertions, deletions and replacements needed to turn one string into another [23]. Navarro's survey describes many ways of approximate string matching and their uses [24]. The RapidFuzz library offers fast versions of these ideas in Python [25]. We use its WRatio scorer, which tries several comparison styles and keeps the best weighted score between 0 and 100.

Our application follows the usual client-server style in which the browser talks to a server through HTTP requests and receives JSON responses, in the manner of REST [26]. The front

end is built with React [27], the main server is build with Node.js and Express [28], the AI serveris build with Flask [29], and the datas are stored in MongoDB [30].

Table 1 puts the closest related works side by side with our approach.

Table 1: Comparison of related work and proposed system

Work	Main idea	Needs special training?	Web app / history?
Paul et al. [11]	Find medicine lines, add medicine language model	Yes (synthetic data)	Not reported
Ali et al. [12]	Mask R-CNN + TrOCR, then database matching	Yes (fine-tuning)	Not reported
Mankash et al. [13]	Fine-tuned multimodal LLMs on simulated records	Yes (very large data)	Not reported
Proposed system	OpenCV + PaddleOCR + Gemini + RapidFuzz name check	No (ready-made models)	Yes (React, Express, MongoDB)

A common problem found is that many papers are concerned with achieving the highest levels of recognition accuracy. This means that there are less fully working and easy to use applications which take an image from a standard user and deliver a verified structured output with a record of their history. Our project attempts to overcome this with a simple pipeline which does not require any training of models and has transparent uncertainty. It highlights uncertain drug names..

3. PROBLEM DEFINITION AND OBJECTIVES

Prescriptions are mostly available only as paper or as photos. There are also issues with spelling errors in the drug name due to OCR. Finally, the data isn't structured in any meaningful way. We need a tool that will take an image of a prescription, extract the information, structure it and highlight any questionable drugs whilst also keeping a history. All of this should be easy to follow by a typical user.

This tool aims to help users read and structure their prescriptions. It is not providing any medical advice or suggestions for treatment. The output page always asks the user to double-check against the original prescription. In Phase-I we will be using images (JPG/PNG), English text and single pages.

Table 2: State of the work at the end of Phase-I

No.	Work item	Status
1	Prescription image upload (React + Express)	Completed
2	OpenCV image preprocessing	Completed
3	PaddleOCR text extraction	Completed
4	Gemini-based prescription structuring	Completed
5	Medicine-name matching with RapidFuzz	Completed
6	MongoDB prescription history	Completed
7	User login and editing of results	Planned for Phase-II
8	Large-scale testing and accuracy measurement	Planned for Phase-II

4. PROPOSED METHODOLOGY AND WORK DESCRIPTION

Overview of the approach

We have broken this down into smaller components and used the appropriate tool for each. For example, we have used OpenCV to clean the image, PaddleOCR to extract the text, Gemini to understand and structure the text. RapidFuzz has been used to check the drug names and MongoDB to store the result. We have created a React website to interact with the user. This way we can minimise errors from each stage by using further stages after. Any uncertain parts are shown to the user.

System architecture

The system has three layers, as shown in Fig. 1. The presentation layer is the React website that runs in the browser. The application layer is a Node.js server built with Express, which receives uploads and talks to the database. The AI layer is a separate Python server built with Flask, which does the image processing, OCR, language-model call and name checking. We kept the AI part in Python because PaddleOCR, OpenCV and RapidFuzz are Python libraries, and we kept the web part in Node.js because it is easy to build and connect with React.

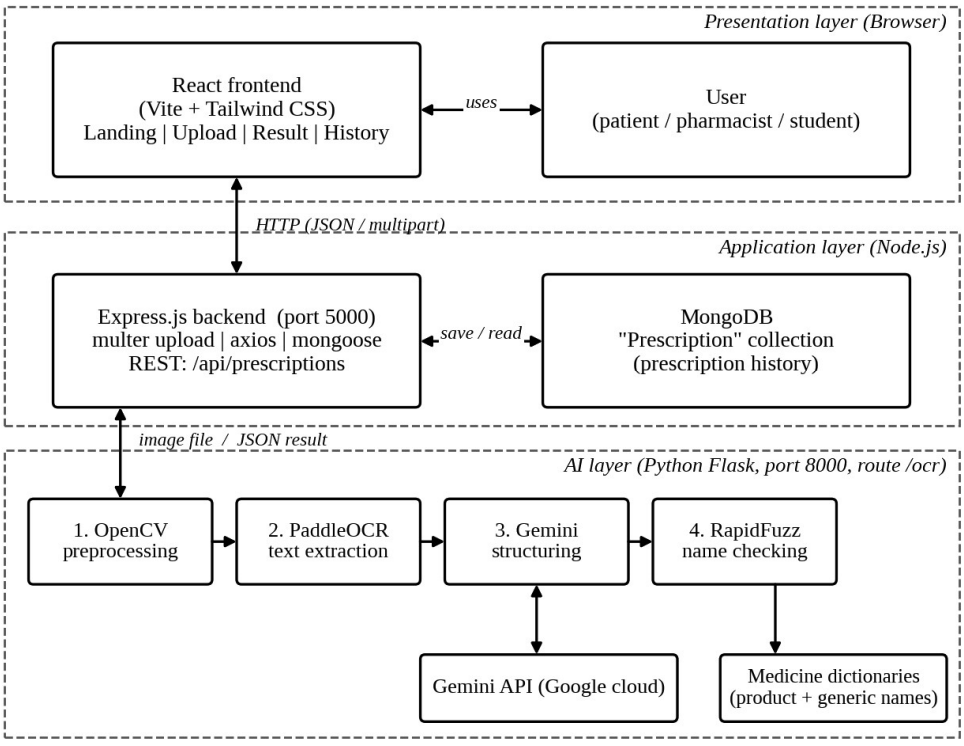


Fig. 1: Architecture of Medical Prescription AI

Step-by-step working

1. The user selects a file on the Upload page. The page checks that the file is a JPG, PNG or PDF and that it is not larger than 10 MB. If the check fails, an error message is shown and nothing is sent. Otherwise the file is sent to the Express server as a form upload.

2. The file is saved in the uploads directory with a timestamped filename. It is then passed onto the /ocr endpoint on the Flask server.

3. On the Flask server the file is saved as a temporary file. The image is then processed using OpenCV using the following steps: convert to grey scale, apply CLAHE to increase local contrast and sharpen the writing. See section 7.5.

4. The processed image is then passed onto PaddleOCR to extract the text. Each line of text is located and extracted. It is then combined to create the OCR text.

5. This text is passed onto a Gemini model using the following instruction. We want the model to output a JSON containing patients name, doctors name, list of medications (with name, dose, frequency, duration), and list of instructions. If any of these are missing you should put 'null'. If uncertain, you should put 'uncertain'. The prompt is clear that we do not want guesses or made up med names.

6. For each medication output by Gemini we check the medication name against the medication dictionaries (section 4.4). This is then saved in the name_review field with the medication. The original name, suggested name, match type, similarity score and needs_review boolean are all recorded.

7. The OCR text and structured prescription are returned to Express by the Flask server. The record is saved into MongoDB and returned to the browser. The Result page displays this information. Temporary image files on the AI server are deleted at the end of each request.

Medicine datasets and name checking

For name checking we use two medicine datasets in CSV form. The first has 195,605 rows and 8 columns (including product name, salt composition and manufacturer). The second has 11,498 rows and 7 columns (including medicine name, composition and form). From them we prepared two lists of names stored as JSON files, one with product (brand) names and one with generic names.

Table 3: Datasets and name lists generated

Item	Size	Use in the project
medicine_data.csv	195,605 rows, 8 columns	https://www.kaggle.com/datasets/mohneesh7/indian-medicine-data
medicine_dataset.csv	11,498 rows, 7 columns	https://www.kaggle.com/datasets/ujjwalaggarwal402/medicine-dataset
product_names.json	16,449 names	Brand names used for matching
generic_names.json	1,305 names	Generic names used for matching

The matching works as shown in Fig. 2. The medicine name is compared with all product names and with all generic names using the WRatio scorer of RapidFuzz. The best match from each list is taken and the higher of the two scores is kept. If this score is 80 or more, it is shown as a suggestion together with its type (product or generic). If it is below 80, no suggestion is shown and the name is marked for review. The name is also marked for review whenever the suggestion is spelled differently from the original, so that the user always looks at a corrected name before trusting it.

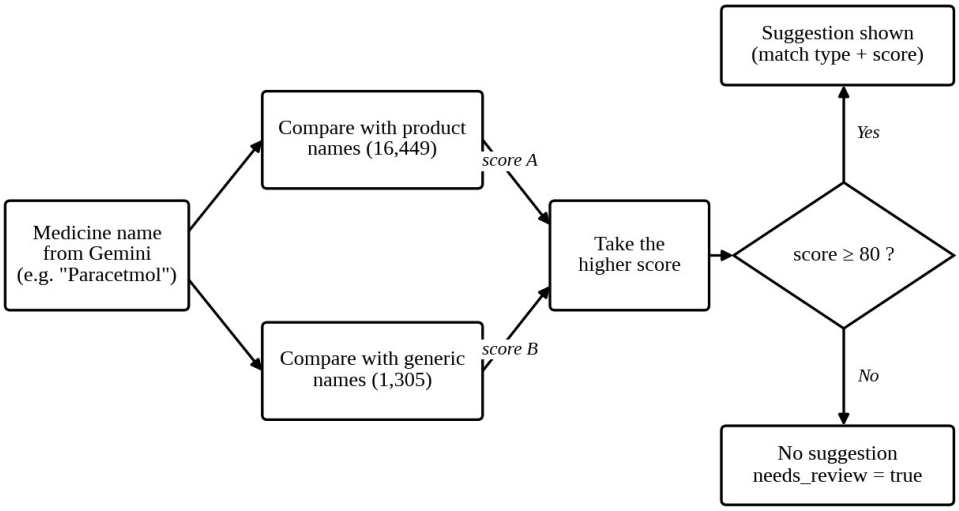


Fig. 2: Medicine name checking with RapidFuzz

Important design decisions

The name is suggested rather than changed. The suggestion is given in brackets next to the original name. This is a safer option as incorrect automated changes could be dangerous within a medical context.

No models have been trained. All models used are pre-trained. This enabled the project to be completed in Phase-I and kept things simple to set up.

No guessing. The language model has been forced to use null and uncertain rather than guess.

Python section has been separated. This means the Python section can be altered or moved to another computer without affecting the website.

Warning on results page. There is a warning on the Results page that mistakes may occur with the OCR and AI. The results should always be double-checked.

Data stored for each prescription

Each processed prescription is saved as document in MongoDB. Fig. 3 and Table 4 show its fields. The medicines are kept as a list inside the prescription, and each medicine have its own name_review object.

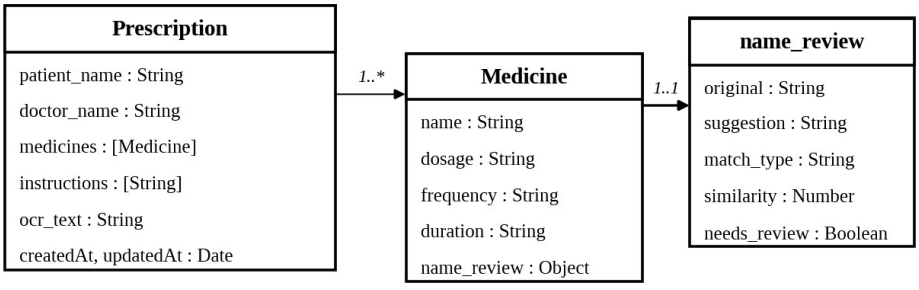


Fig. 3: Data model of a saved prescription

Table 4: Prescription fields that will be stored

Field	Type	Meaning
patient_name	String	Patient name found in the prescription (or null)
doctor_name	String	Doctor name found in the prescription (or null)
medicines	List	Each item has name, dosage, frequency, duration and name_review
instructions	List of strings	Extra advice written in the prescription
ocr_text	String	Raw text read by PaddleOCR
createdAt, updatedAt	Date	Added automatically by Mongoose

5. PROPOSED ALGORITHMS

This chapter gives the algorithm of website flow in the simple steps.

Algorithm 1: End-to-end prescription processing

Input: prescription image document. Output: structured prescription .

1. The user selects a image file and put in the website to get the result.
2. If the file type is not JPG, PNG or PDF, or the size is more than 10 MB, it will show an error and stop.
3. The file is sent to the Express server (POST /api/prescriptions/upload).
4. Express saves the file and forwards it to the Flask server (POST /ocr).
5. Flask saves a temporary copy and calls PreprocessImage (Algorithm 2).
6. Give the processed image to PaddleOCR and collect the recognised text lines.
7. Join the lines into one OCR text.
8. Send the OCR text to Gemini and get a JSON object with patient, doctor, medicines and instructions.
9. For each medicine, call SuggestName (Algorithm 3) and store the result as name_review.
10. Delete the temporary files.
11. Express saves the OCR text and the structured data in MongoDB.
12. Return the result to the browser and show it on the Result page.
13. If any step fails, return an error message and show it to the user.

Algorithm 2: PreprocessImage

Input: path of the image. Output: path of the cleaned image.

1. Read the image. If it cannot be read, raise an error.
2. Convert the image to grayscale.
3. Apply CLAHE with clip limit 2.0 and an 8 x 8 tile grid.
4. Blur the result with a Gaussian filter (sigma = 3).
5. Sharpen: output = 1.5 x enhanced image - 0.5 x blurred image.
6. Save the output image and return its path.

Algorithm 3: SuggestName (medicine name checking)

Input: medicine name text, threshold T = 80. Output: original, suggestion, match type, score, needs_review.

1. If the name is empty, return no suggestion and needs_review = true.
2. Find the best match of the name in the product-name list using WRatio.
3. Find the best match of the name in the generic-name list using WRatio.
4. Keep the match with the higher score (round it to one decimal).
5. If score >= T, accept it as the suggestion. Otherwise there is no suggestion.
6. Set needs_review = true if there is no suggestion.
7. After this, in the AI server, also set needs_review = true when the suggestion is spelled differently from the original name.
8. Return the result.

6. PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

This chapter shows the block diagram, the flowchart and the data flow diagrams of the system. The block diagram of the architecture is already given in Fig. 1 in Chapter 4.

Flowchart

Fig. 4 shows how a prescription moves through the system from the moment the user selects a file to the moment the result is shown.

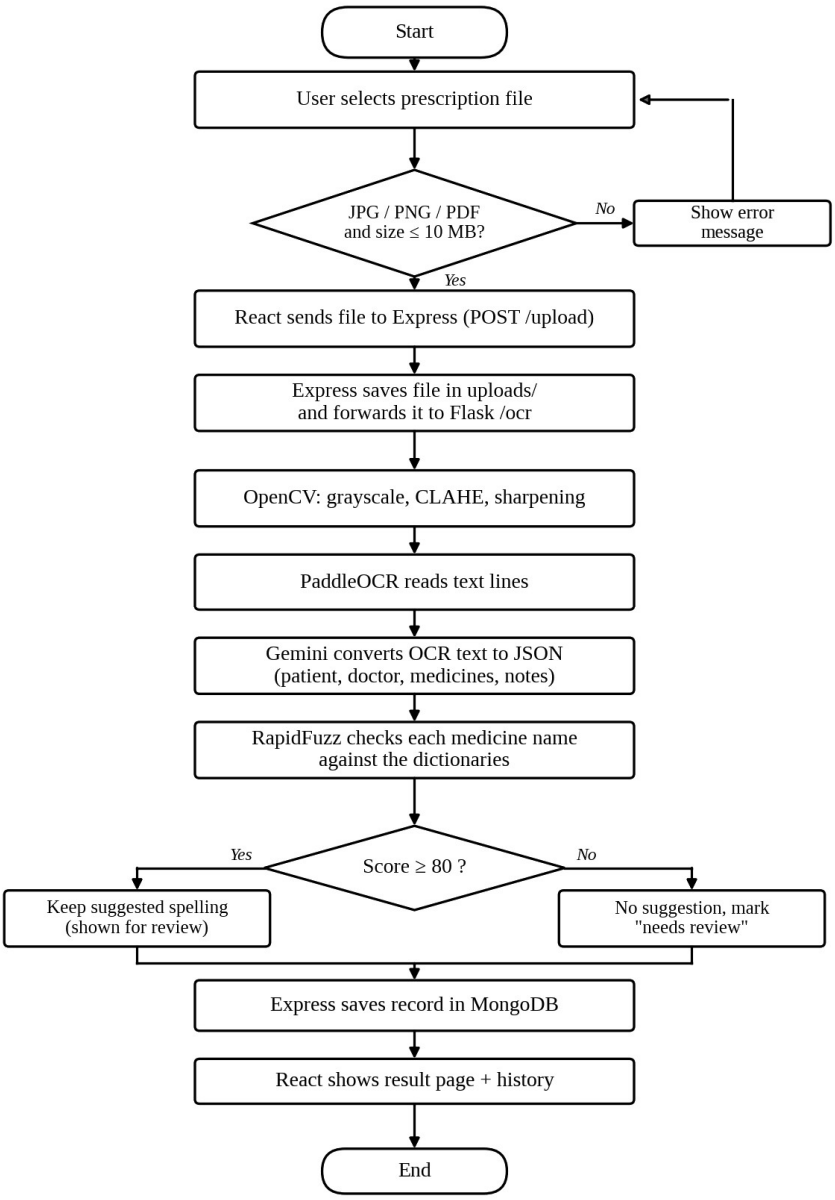


Fig. 4: Flowchart of prescription processing

Data flow diagram (Level 0)

The Level 0 diagram in Fig. 5 shows the whole system as one process. The user uploads a prescription image and gets a structured , well mannered result. The system exchanges OCR text and JSON with the Gemini API, and saves and reads records in MongoDB.

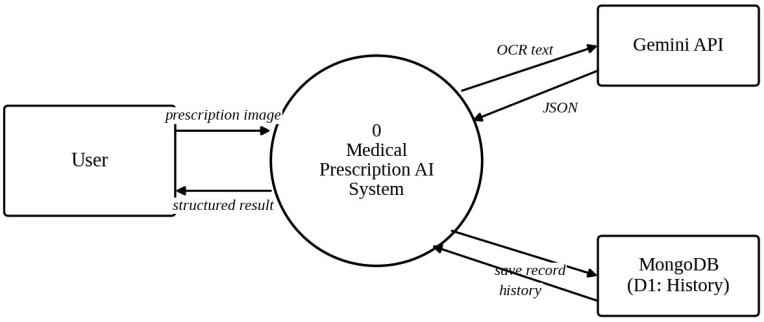


Fig. 5: Data flow diagram, Level 0

Data flow diagram (Level 1)

Fig. 6 it splits a process into six smaller processes. D1 is used to uploads folder on the Express server, D2 for the set of medicine dictionaries, and D3 for the MongoDB history.

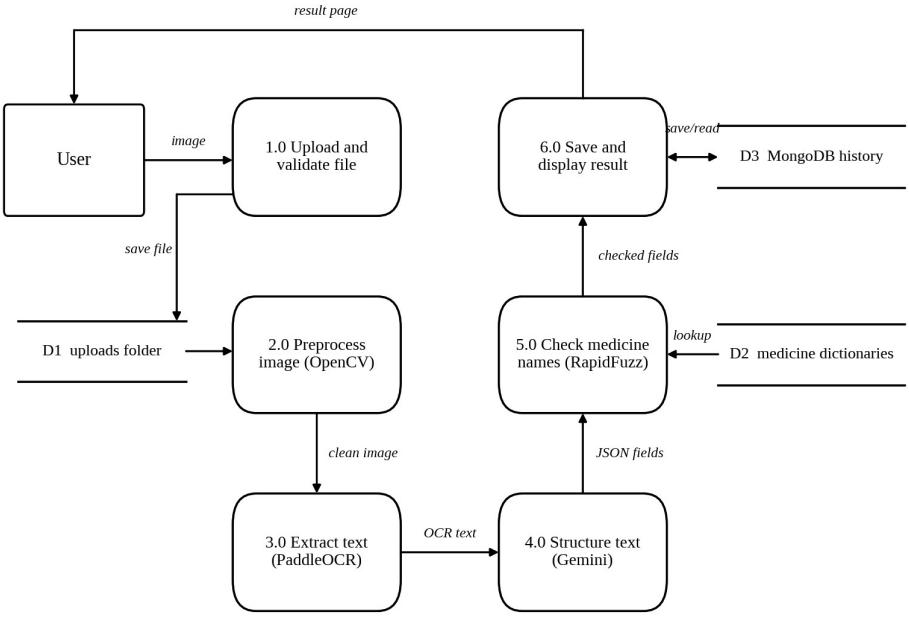


Fig. 6: Data flow diagram, Level 1

7. IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

Tools and technologies

Table 5: Tools and technologies used

Layer	Tool	Purpose
Frontend	React 19, Vite, React Router	Pages and navigation
Frontend	Tailwind CSS 4, lucide-react	Styling and icons
Frontend	Axios	Calling the backend API
Backend	Node.js, Express 5	REST API and file receiving
Backend	Multer, form-data, Axios	Upload handling and forwarding to Flask
Backend	Mongoose 9, MongoDB	Saving and reading prescriptions
AI server	Python, Flask	Serving the /ocr route
AI server	OpenCV	Image preprocessing
AI server	PaddleOCR	Text detection and recognition
AI server	Gemini (google-genai)	Structuring the OCR text
AI server	RapidFuzz	Medicine name matching
AI server	python-dotenv	Reading secret keys from .env

Frontend

The frontend is a single-page React application with four sections.

Landing page: which explains the idea in three steps (upload, AI processing, review and save), lists benefits and shows a medical disclaimer.

Upload page: that lets the user pick or drag a file. It shows a small preview for images, checks the file type (JPG, PNG, PDF) and size (up to 10 MB), and shows an error banner if something goes wrong. While the file is being processed the button shows that work is in progress.

Result page: it shows a warning to verify the output, the raw OCR text, the patient and doctor names, and every medicine with its dosage, frequency and duration. For a medicine with a name_review it shows the original name, the suggested spelling and the similarity percentage, or a message that no close match was found.

History page: it does loads all saved prescriptions from the server, newest first. Each card shows the patient name, the doctor name, the number of medicine entries and the date, with a button to open it in the Result page.

The top bar of every page has Log in and Register buttons, but user accounts are not working in Phase-I and are planned for Phase-II. In the same way, the landing page speaks about reviewing and correcting the result; editing of the result is also planned for Phase-II.

The Vite development server forwards every request that starts with /api to the Express server on port 5000, so the frontend code only has to call /api/... paths.

Node.js and Express backend

The Express server listens on port 5000. It uses cors so that the browser can call it, and Multer to save uploaded files in an uploads folder with the current time added to the file name. It exposes two routes, shown in Table 7. After receiving a file it sends it to the Flask server with Axios, waits for the answer, saves the prescription with Mongoose, and replies to the browser. The server starts only after MongoDB is connected, and it exits with a message if the connection fails.

Python AI server

The Flask server listens on port 8000 and has one route, POST /ocr. The PaddleOCR object is created once when the server starts (English language, with text-line orientation detection switched on), so it is not loaded again for every request. For each request the server saves a temporary copy of the file, runs the pipeline, builds the JSON answer, and removes the temporary files in a finally block, so the files are deleted even when an error occurs. If no file is sent the server replies with status 400, and if processing fails it replies with status 500 and an error message.

Image preprocessing in OpenCV

The preprocessing is done in a small function. It is intentionally short, because each step was chosen to help OCR without damaging thin pen strokes. The settings are given in Table 6.

Table 6: Preprocessing steps and settings

Step	OpenCV function	Setting	Why
Grayscale	cvtColor	BGR to gray	Colour is not needed for reading text
Contrast	createCLAHE	clip limit 2.0, tiles 8 x 8	Brings out faint writing and reduces the effect of shadows
Sharpening	GaussianBlur + addWeighted	sigma 3, weights 1.5 and -0.5	Makes the edges of the handwriting clearer

The result is saved as a grayscale image. The current version does not use thresholding to a pure black and white image, and it does not resize or denoise the picture. These can be tried later.

Text extraction with PaddleOCR

PaddleOCR is called with the processed image. For each image it returns the recognised text lines, and we collect them in the order in which they are found. The lines are joined with a new line to form the OCR text that is shown to the user and sent to Gemini.

Structuring with Gemini

The OCR text is placed inside a short prompt and sent to a Gemini Flash-Lite model (set as gemini-3.5-flash-lite in llm.py). The request asks for a JSON response, and the program converts the answer into a Python object. The prompt says that the model should only organise

what is present in the OCR text, should not guess or invent medicine names, should write "uncertain" for unclear items and null for missing items, and should return only valid JSON in a fixed format. The key for the Gemini service is kept in an environment file (.env) and is never written inside the code.

Medicine name checking with RapidFuzz

The two name lists are loaded from JSON files once at the start. The function suggest_medicine_name compares a name with both lists using RapidFuzz's extractOne function and the WRatio scorer, and applies the threshold of 80 as explained in Section 4.4. The function returns the original name, the suggestion, the match type, the score and the needs_review flag. A small wrapper in the AI server adds this result to every medicine as name_review.

Database

Our prescription model is written with Mongoose. It holds the patient name, doctor name, the list of medicines (each with name, dosage, frequency, duration and name_review), the instructions, the OCR text and automatic created and updated times. Prescriptions are read in reverse order of creation, so the newest one comes first in the history page.

API routes

Table 7: Routes of the system

Server	Method and route	Purpose
Express (port 5000)	POST /api/prescriptions/upload	Receive the file, call the AI server, save and return the result
Express (port 5000)	GET /api/prescriptions	Return all saved prescriptions, newest first
Flask (port 8000)	POST /ocr	Preprocess, read, structure and check the prescription

Running the system

Three programs are started separately: the Flask AI server (python ai_server.py), the Express server (node server.js), and the React development server (npm run dev). MongoDB must be reachable through the connection string in the server's .env file, and the Gemini key must be present in the AI server's .env file.

Safety and privacy points

Secret keys are kept in .env files and not in the code. The Result page shows a verification warning. Temporary files created by the AI server are removed after every request. At the same time, there are things that are not done yet: there is no user login, uploaded images stay in the uploads folder of the Express server, and the file type and size are checked only in the browser. These points are listed in the future scope.

8. RESULT DISCUSSION AND ANALYSIS

This part presents what we have observed from the prototype. In Phase-I we did not have a large labelled set of prescriptions, so the results below are small, controlled checks that show how each part behaves. The required spaces are kept for the full measurements that will be added when they are ready.

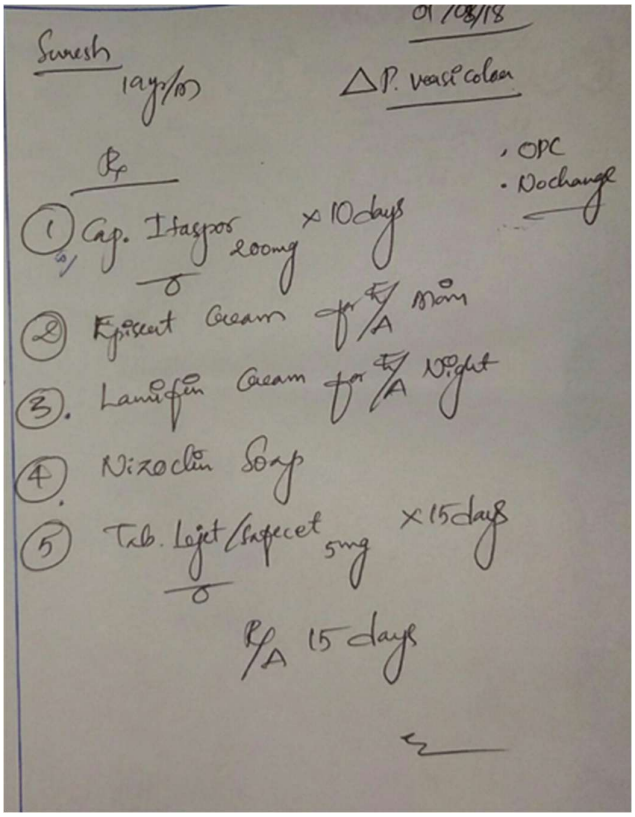
Test environment

Table 8: Test environment

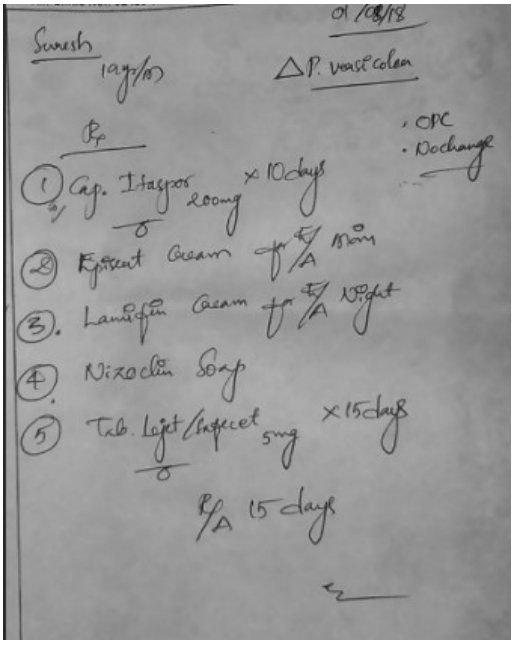
Item	Details
Computer / CPU	Intel Core i5-1235U
RAM	16 GB
Operating system	Windows 11 64-bit
Python / Node.js version	Python 3.13.5 / v22.16.0
Browser used	Google Chrome

Effect of image preprocessing

Fig. 7 shows a part of a handwritten prescription photo before and after our preprocessing (the name and contact lines at the top of the page are cropped out). In the original photo the paper looks grey and the ink is low in contrast. After preprocessing the background is more even, the ink is darker, and the strokes of the handwriting stand out more clearly. The result is still grayscale, so thin lines are not lost.



Before preprocessing



After preprocessing

Fig. 7: A sample prescription before and after preprocessing

Sample OCR output

Fig. 7 is the same sample that we ran through the complete system. PaddleOCR read 23 lines from it. Table 9 lists them, except the first two lines (the date and the patient's name), which are left out. The handwriting is difficult, and the output shows it. Only a few words, such as "Nizoclin" and "Lanifin", are close to what is written on the page. The first two medicines come out as "Qap. Iageor" and "Esicant Geamn", and many lines are only one character or a symbol (for example 8, 2, D, A, + and 3), most likely from the circled serial numbers and the marks on the page. This is the reason why a name-checking step is needed after OCR.

Table 9: OCR lines from a sample handwritten prescription

Line	Text read by OCR	Line	Text read by OCR
3	lagython	14	Caeam
4	P. veasi colaan	15	toavgt
5	8	16	.Lanifin
6	,OPC	17	soy
7	2	18	Nizoclin
8	Qap. Iageor	19	+
9	. Nochaw	20	hs p
10	Loong	21	xi5clays
11	D	22	Ha 15day
12	© Esicant Geamn	23	3
13	A		

Medicine name checking test

We tested the name-checking function with 18 typed inputs. They include common spelling mistakes of well-known medicines, brand names with strengths, names taken from the sample prescription, and two inputs that are not medicine names at all. We then looked at the output and marked it as right or wrong. The results are in Table 10.

Table 10: Result of the medicine-name checking test (threshold = 80)

No.	Input name	Suggested name	Type	Score	Outcome
1	Paracetmol	Paracetamol	Generic	95.2	Correct
2	Azithromicin	Azithromycin	Generic	91.7	Correct
3	Pantoprazol	Pantoprazole	Generic	95.7	Correct
4	Cetrizine	Cetirizine	Generic	94.7	Correct
5	Metformn	Metformin	Generic	94.1	Correct
6	Ibuprofin	Ibuprofen	Generic	88.9	Correct
7	Itraconazole	Itraconazole	Generic	100.0	Correct
8	Dolo 650	Dolo 650 Tablet	Product	90.0	Correct
9	Crocin	Crocin Advance 500mg Tablet	Product	90.0	Correct

No.	Input name	Suggested name	Type	Score	Outcome
10	Lejet	Lejet Tablet	Product	90.0	Correct
11	Nizoclin Soap	Nizoclin Soap	Product	100.0	Correct
12	Nizoclin	Nizoclin Cream	Product	90.0	Correct
13	Lamifen Cream	Lamifin Cream	Product	92.3	Correct
14	Xyzabc	None	-	72.0	Correctly rejected
15	Qap lageor	None	-	56.2	Correctly rejected
16	Lanifin	None	-	77.1	Missed (below 80)
17	Amoxicilin 500	5 Flucel 500 Injection 10ml	Product	85.5	Wrong match
18	Episcat Cream	Acnetreat 0.05% Cream	Product	85.5	Wrong match

Out of the 18 inputs, 13 got a useful suggestion, 2 were correctly rejected, 1 was missed because its score (77.1) was just below the threshold, and 2 got a wrong suggestion. So this took the right decision in 15 of 18 cases. This is a very small test with inputs chosen by us, so it shows how it behaves and should not be taken as the overall accuracy of the system.

From the followings two wrong matches have one thing in common. Both the inputs contained extra words or numbers ("500" and "Cream") that also appear in many unrelated product names, and WRatio gave these a high score. This tells us that the dosage and the form words should be removed from the name before matching. We have noted this as an improvement for Phase-II.

Screens of the application and a full run

Figures 8 to 11 show the four pages of the web application. The names of patients and doctors in the screenshots are hidden.

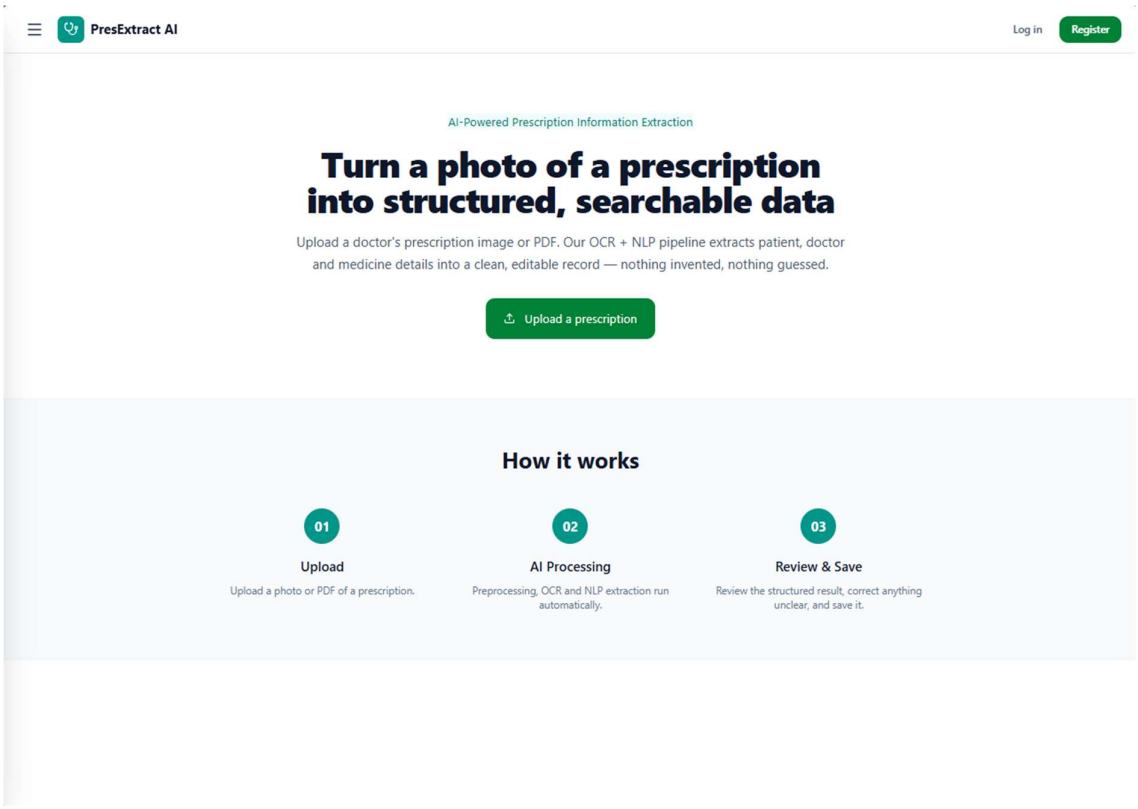


Fig. 8: Landing page of the application

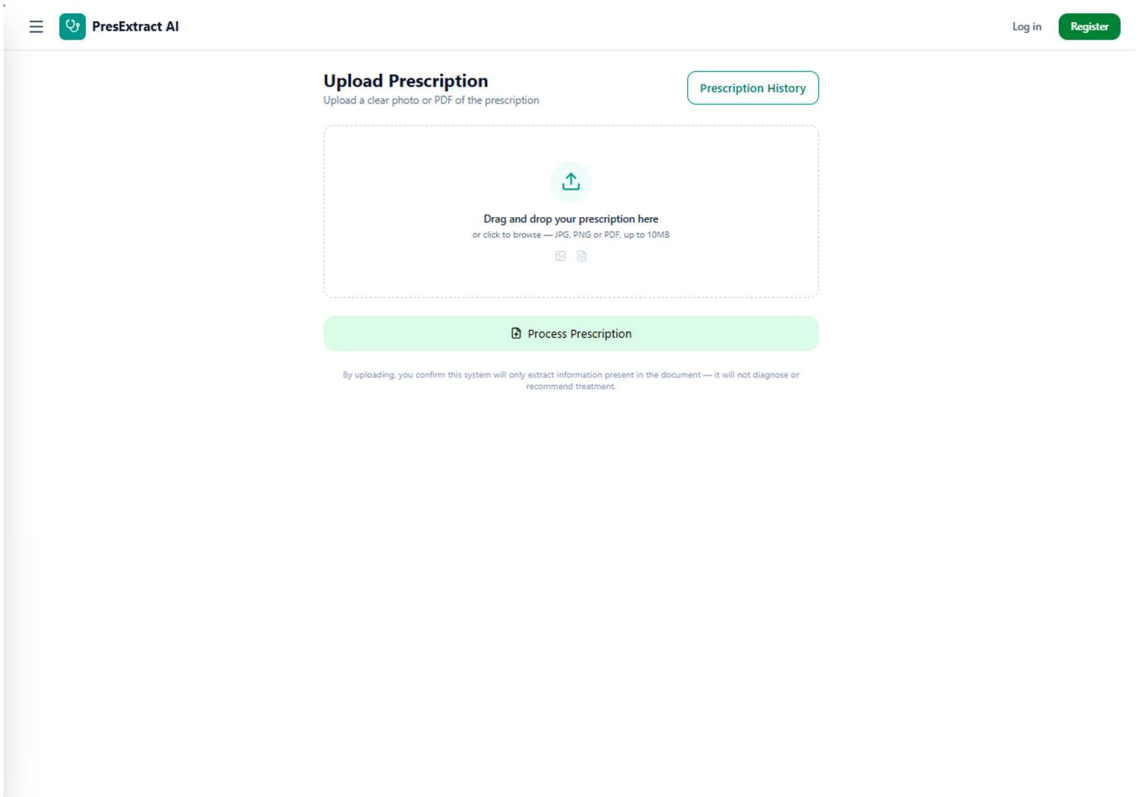
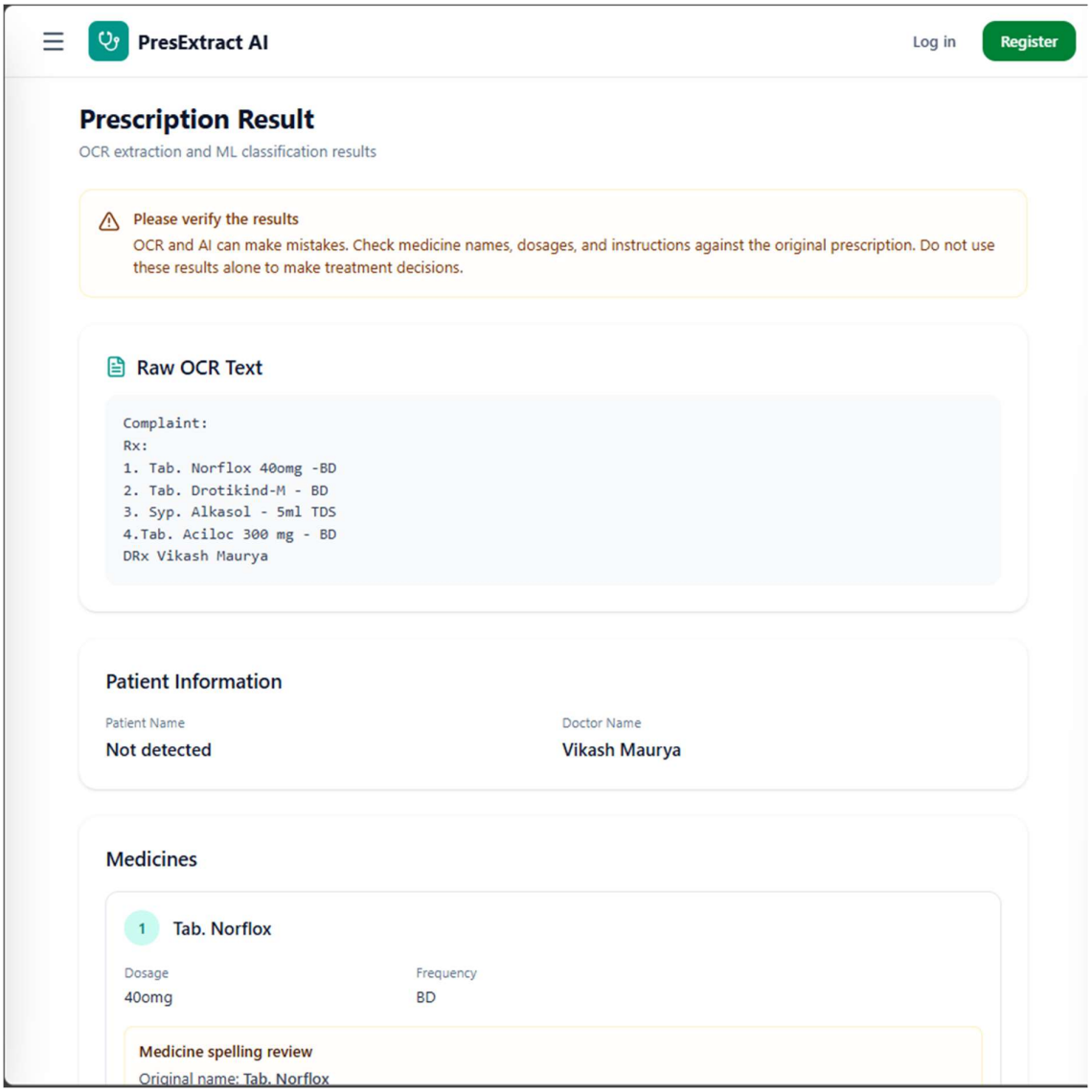


Fig. 9: Upload page

Fig. 10 shows what the Result page displayed after the sample prescription of Fig. 7 was uploaded. The page starts with the verification warning and the raw OCR text, then shows the patient and doctor details, and then one card for each medicine entry. Every card has a "Medicine spelling review" box that shows the original name and either a suggested spelling with its similarity or the message "No reliable spelling match found".


PresExtract AI
Log in
Register
Prescription Result
OCR extraction and ML classification results
Please verify the results
OCR and AI can make mistakes. Check medicine names, dosages, and instructions against the original prescription. Do not use these results alone to make treatment decisions.
Raw OCR Text
Complaint:
Rx:
1. Tab. Norflox 40omg -BD
2. Tab. Drotikind-M - BD
3. Syp. Alkasol - 5ml TDS
4.Tab. Aciloc 300 mg - BD
DRx Vikash Maurya
Patient Information
Patient Name
Not detected
Doctor Name
Vikash Maurya
Medicines
1
Tab. Norflox
Dosage
40omg
Frequency
BD
Medicine spelling review
Original name: Tab. Norflox

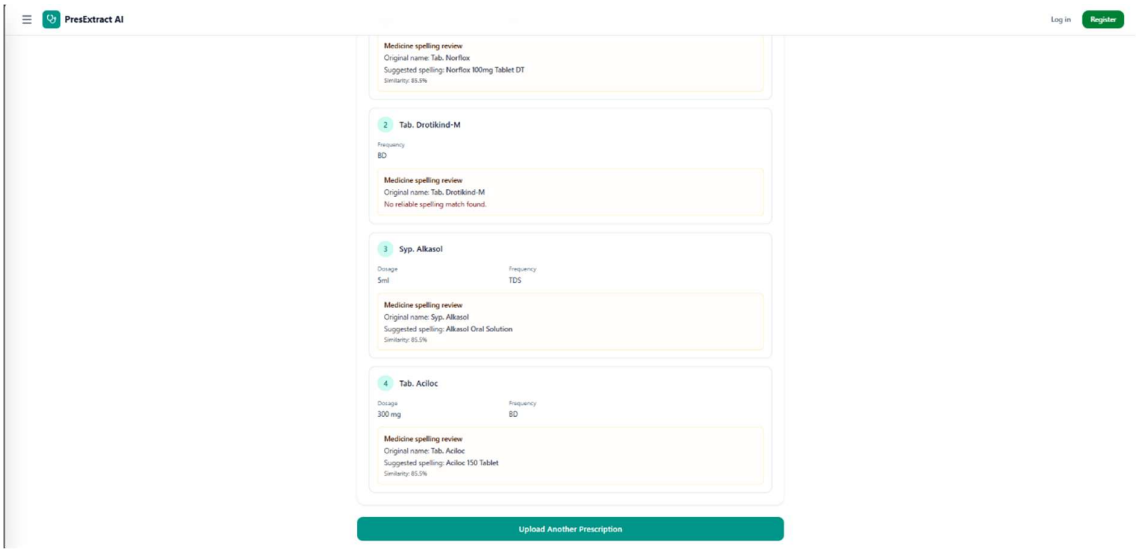

PresExtract AI
Log in
Register
Medicine spelling review
Original name: Tab. Norflox
Suggested spelling: Norflox 100mg Tablet DT
2
Tab. Drotikind-M
Frequency
BD
Medicine spelling review
Original name: Tab. Drotikind-M
No reliable spelling match found.
3
Syp. Alkasol
Dosage
5ml
Frequency
TDS
Medicine spelling review
Original name: Syp. Alkasol
Suggested spelling: Alkasol Oral Solution
4
Tab. Aciloc
Dosage
300 mg
Frequency
BD
Medicine spelling review
Original name: Tab. Aciloc
Suggested spelling: Aciloc 150 Tablet
Upload Another Prescription

Fig. 10: Result page for the sample prescription

In this run Gemini returned nine medicine entries, although the handwritten page has five numbered items. Several entries, such as "lagython", "P. veasi colaan" and "Nochaw", look like misread words from the header or the notes and not like medicine names. The name check found no reliable match for seven of the nine entries and flagged all of them for review. It gave a suggestion for two entries: "Nizoclin" became "Nizoclin Cream" (90%), and "Loong" became "Long Drive 30mg Tablet" (80%), which does not look right to us and sits exactly on the threshold. So in this sample the weakest points are the OCR on handwriting and the language model treating noise as medicines. The warning and the review boxes are what protect the user from trusting these entries.

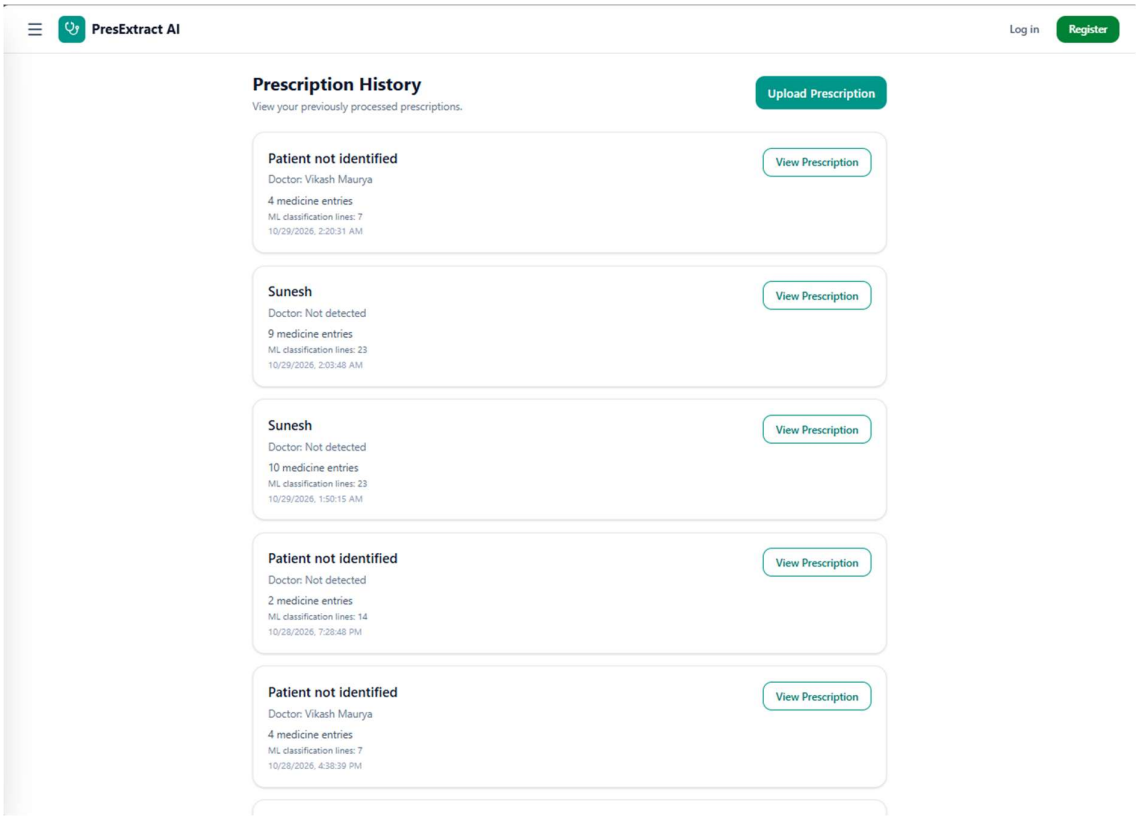


Fig. 11: Prescription history page

Comparison with other ways of handling prescriptions

Table 11 compares our pipeline with other simple ways of handling a prescription image. The comparison is about the features of each approach, not about accuracy numbers.

Table 11: Feature comparison of approaches

Feature	Typing by hand	OCR only	OCR + LLM only	Proposed system
Works from an image	No	Yes	Yes	Yes
Structured fields	Yes (manual)	No	Yes	Yes
Checks medicine names	By the typist	No	No	Yes
Flags doubtful items	No	No	Partly	Yes
Keeps history	Depends	No	No	Yes
Needs model training	No	No	No	No
Effort per prescription	High	Low	Low	Low

Accuracy and speed measurements

The full measurement on a set of real prescriptions is not part of Phase-I. Table 12 is kept ready for these numbers.

Table 12: Measured accuracy and time (to be filled)

Prescription type	No. of samples	Lines read correctly	Medicine names correct before check	Medicine names correct after check	Average time (s)
Printed	10	Not measured in Phase-I	Not measured	Not measured	Not measured
Clear handwriting	6	Not measured	Not measured	Not measured	Not measured
Difficult handwriting	5	Not measured	Not measured	Not measured	Not measured

Discussion

The results of these checks highlight a few important points about the system. First, image preprocessing improves the quality of the prescription image, making it easier for the OCR system to read the text. However, handwritten prescriptions are still difficult to process, as OCR often makes mistakes when recognising handwritten words. Similar challenges have also been reported in earlier studies [11,13].

The medicine-name verification feature helps identify common spelling errors in familiar medicine names. However, it does not always give accurate matches when additional details, such as medicine strength or dosage form, are included in the extracted name. Another issue was noticed while processing the sample prescription in Section 8.5. In some cases, the language model treated incorrectly recognised words as medicine names. This shows the need for an additional filtering step to separate medicine-related information from other text.

Overall, the results suggest that the application can be useful as a supporting tool for reading prescriptions. Since users can see the extracted information and review uncertain results, the system assists them without making final medical decisions on their behalf.

The main limits of the current prototype are: it was tested only on a few samples; handwritten text is often misread by OCR; it depends on the quality of the photo; the Gemini step needs an internet connection; PDF files are accepted by the upload form but the image-processing code reads image files only, so PDF handling is not complete; and there is no login, so all saved prescriptions are in one common history.

9. CONCLUSION AND FUTURE SCOPE

Conclusion

As part of this project, so far we have developed Medical Prescription AI, a web application that takes a prescription image and converts the information into an organised record. The prototype brings together a React-based website, a Node.js and Express server, a Python AI service and a MongoDB database. On the AI side, OpenCV is used to improve the image, PaddleOCR extracts the text, and Gemini helps us to arrange the extracted information into useful fields. The names of the medicines are then checked with RapidFuzz against two medicine datasets.

During development of our project , we found that relying on just one technique would not be enough to handle prescription reading . OCR can misread unclear handwriting, a language model may interpret text incorrectly, and string matching cannot understand the full meaning of a prescription. Combining these tools gives the system a more useful way to process the information. Showing the original medicine name alongside a suggested match, and a warning also helps users review the result instead of accepting it without any checking. Our initial tests highlighted that medicine name matching still has limitations. These findings will help guide the improvements planned for the next phase.

Future scope :-

Hoping we would made a prototype that can perform following operations:-

Before comparing medicine names, remove details such as strength and dosage forms (for example, 500 mg, cream or tablet). This may help prevent incorrect matches like the cases shown in Table 10.

Evaluating the system using a larger collection of both printed and handwritten prescriptions. Record its accuracy, error rate and processing time, and present the results in Table 12.

Explore handwriting recognition models, such as TrOCR [10], to improve the reading of prescription lines that contain medicine names.

Improve PDF handling so that users can upload multi-page documents. The server should also check the uploaded file type and size.

Introduce user accounts and private prescription histories. Users should also be able to delete uploaded images so that they have better control over their personal information.

Allow users to review, edit and confirm the extracted details before saving a prescription.

Reduce unrelated entries in the output. For example, the language model could be instructed to return only text that appears to be medicine-related, while entries that do not match the medicine databases could be flagged or excluded for review.

Recognise common dosage abbreviations, such as 1-0-1 and BD, and explain them in simple language.

Use the drug-interaction information available in the dataset to flag medicine combinations that may need further checking by a doctor or pharmacist.

Add support for regional languages and prepare the application for straightforward deployment.

REFERENCES

[1] L. T. Kohn, J. M. Corrigan, and M. S. Donaldson, Eds., To Err Is Human: Building a Safer Health System. Washington, DC, USA: National Academies Press, 2000.

[2] World Health Organization, "Medication Without Harm: WHO Global Patient Safety Challenge," WHO, Geneva, Switzerland, 2017.

[3] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), Curitiba, Brazil, 2007, pp. 629–633.

[4] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 11, pp. 2298–2304, Nov. 2017.

[5] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in Proc. 23rd Int. Conf. Machine Learning (ICML), Pittsburgh, PA, USA, 2006, pp. 369–376.

[6] A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke, and J. Schmidhuber, "A novel connectionist system for unconstrained handwriting recognition," IEEE Trans. Pattern Anal. Mach. Intell., vol. 31, no. 5, pp. 855–868, May 2009.

[7] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarization," in Proc. AAAI Conf. Artificial Intelligence, vol. 34, no. 7, 2020, pp. 11474–11481.

[8] Y. Du et al., "PP-OCR: A practical ultra lightweight OCR system," arXiv:2009.09941, 2020.

[9] C. Cui et al., "PaddleOCR 3.0 technical report," arXiv:2507.05595, Jul. 2025.

[10] M. Li et al., "TrOCR: Transformer-based optical character recognition with pre-trained models," in Proc. AAAI Conf. Artificial Intelligence, vol. 37, no. 11, 2023, pp. 13094–13102.

[11] S. Paul, G. Madan, A. Mishra, N. Hegde, P. Kumar, and G. Aggarwal, "Weakly supervised information extraction from inscrutable handwritten document images," arXiv:2306.06823, Jun. 2023.

[12] U. Ali, S. Ranmbail, M. Nadeem, H. Ishfaq, M. U. Ramzan, and W. Ali, "Leveraging deep learning with multi-head attention for accurate extraction of medicine from handwritten prescriptions," arXiv:2412.18199, Dec. 2024.

[13] T. Mankash, V. S. C. Kota, A. De, P. Prakash, and K. Jadhav, "MIRAGE: Multimodal identification and recognition of annotations in Indian general prescriptions," arXiv:2410.09729, Oct. 2024.

[14] S. M. Pizer et al., "Adaptive histogram equalization and its variations," Computer Vision, Graphics, and Image Processing, vol. 39, no. 3, pp. 355–368, 1987.

[15] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in Graphics Gems IV, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.

[16] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Syst., Man, Cybern., vol. 9, no. 1, pp. 62–66, Jan. 1979.

[17] G. Bradski, "The OpenCV library," Dr. Dobb's Journal of Software Tools, 2000.

[18] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 5998–6008.

[19] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[20] K. Singhal et al., "Large language models encode clinical knowledge," Nature, vol. 620, pp. 172–180, 2023.

[21] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, Art. no. 248, 2023.

[22] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[23] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[24] G. Navarro, "A guided tour to approximate string matching," ACM Computing Surveys, vol. 33, no. 1, pp. 31–88, Mar. 2001.

[25] M. Bachmann, "RapidFuzz: Rapid fuzzy string matching in Python and C++." [Online]. Available: https://github.com/rapidfuzz/RapidFuzz. Accessed: Oct. 2026.

[26] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[27] Meta Open Source, "React documentation." [Online]. Available: https://react.dev. Accessed: Oct. 2026.

[28] OpenJS Foundation, "Express – Node.js web application framework." [Online]. Available: https://expressjs.com. Accessed: Oct. 2026.

[29] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com. Accessed: Oct. 2026.

[30] MongoDB, Inc., "MongoDB manual." [Online]. Available: https://www.mongodb.com/docs/manual/. Accessed: Oct. 2026.

PLAG REPORT (Through Turnitin)

Similarity Plagiarism Report + AI Plagiarism Report